

# ML-DL-Ops Assignment-1

Dwivedi Jyoti Rajeshbhai

M25CSA010

Colab link:

[https://colab.research.google.com/drive/1nj5mUZXJ8MpkcqsdxYaNwKfu0l\\_nnA?usp=sharing](https://colab.research.google.com/drive/1nj5mUZXJ8MpkcqsdxYaNwKfu0l_nnA?usp=sharing)

Github Repository Link:

<https://github.com/Jyoti-Dwivedi-010/MLOps-Jyoti-Dwivedi-M25CSA010>

Github Page link:

<http://jyotidwivedi.me/MLOps-Jyoti-Dwivedi-M25CSA010/>

# 1. Methodology & Engineering Detours

The assignment was about adapting massive architectures designed for complex, colorful photographs (ImageNet) to work efficiently on small, simple grayscale images (MNIST/FashionMNIST). To make this work, I had to make several targeted engineering decisions.

## **1.1 Input Resolution: Why 32×32 instead of 224×224?**

Standard ResNet models are built to ingest large 224 x 224 images. However, the dataset consists of tiny 28 x 28 grids. I faced a choice: drastically upsample the images or adapt the input pipeline.

I chose to resize images slightly to **32×32**.

Because, Upsampling a 28-pixel image to 224 pixels is computationally wasteful. It forces the GPU to perform millions of calculations on what is essentially interpolated "filler" data without adding any actual information. By choosing 32×32 (a standard resolution for CIFAR datasets), I reduced the computational load (FLOPs) by orders of magnitude while ensuring the dimensions were compatible with standard convolutional strides.

## **1.2 Channel Modification: An Adaptation for Grayscale**

ResNet expects a 3-channel RGB image (Red, Green, Blue). The datasets (MNIST & FashionMNIST) are 1-channel grayscale.

I replaced the very first convolutional layer of the network (conv1), changing it from `in_channels=3` to `in_channels=1`. Also, I kept 64 neurons in it with `kernel_size = 7` and a stride of 2 with padding = 3.

Because duplicating the single grayscale channel three times to "fake" an RGB image—is inefficient. It triples the memory used by the input layer for no reason. Modifying the architecture allowed the model to learn directly from the raw data structure without redundant copying.

## **1.3 The "ResNet-32" Customization**

I implemented a custom **ResNet-32**. This architecture is specifically designed for small inputs. It removes the aggressive early pooling layers, maintaining the image's spatial resolution deeper into the network. This allows the model to extract complex features without crushing the data into a single pixel prematurely.

## **1.4 Hardware Optimization: Data for the GPU**

Training speed isn't just about how fast the GPU calculates; it's about how fast the CPU can give it data.

I enabled `pin_memory=True` in the PyTorch DataLoaders.

This allocates the training data in a special "page-locked" area of the CPU's RAM. Without this, the GPU often sits idle waiting for data; with it, data transfer becomes almost seamless, ensuring the training runs were bottlenecked by computation, not data movement.

## 2. Q-1(a) Training a Deep Learning Model

- Datasets: MNIST and FashionMNIST with a 70%-10%-20% train-val-test split.
- Models: ResNet-18 and ResNet-50. (Not pretrained)
- Epochs: 3 and Momentum = 0.9 (for SGD)

A grid search was performed over Batch Sizes (16, 32), Optimizers (SGD, Adam), and Learning Rates (0.001, 0.0001) to identify optimal configurations.

### 2.1 MNIST Results (Test Classification Accuracy)

| Batch Size | Optimizer | Learning Rate | ResNet-18 Acc(%) | ResNet-50 Acc(%) |
|------------|-----------|---------------|------------------|------------------|
| 16         | SGD       | 0.001         | 98.81%           | 98.22%           |
| 16         | SGD       | 0.0001        | 98.26%           | 97.13%           |
| 16         | Adam      | 0.001         | 98.85%           | 98.09%           |
| 16         | Adam      | 0.0001        | 98.08%           | 97.72%           |
| 32         | SGD       | 0.001         | <b>98.93%</b>    | <b>98.36%</b>    |
| 32         | SGD       | 0.0001        | 97.59%           | 95.02%           |
| 32         | Adam      | 0.001         | 98.72%           | 98.15%           |
| 32         | Adam      | 0.0001        | 98.56%           | 96.71%           |

Table-1: MNIST Results

### 2.2 FashionMNIST Results (Test Classification Accuracy)

| Batch Size | Optimizer | Learning Rate | ResNet-18 Acc(%) | ResNet-50 Acc(%) |
|------------|-----------|---------------|------------------|------------------|
| 16         | SGD       | 0.001         | 90.20%           | <b>87.96%</b>    |
| 16         | SGD       | 0.0001        | 88.26%           | 82.49%           |
| 16         | Adam      | 0.001         | 88.89%           | 86.42%           |
| 16         | Adam      | 0.0001        | 89.23%           | 86.04%           |
| 32         | SGD       | 0.001         | 88.38%           | 86.82%           |
| 32         | SGD       | 0.0001        | 87.74%           | 80.21%           |
| 32         | Adam      | 0.001         | <b>89.39%</b>    | 87.04%           |
| 32         | Adam      | 0.0001        | 89.16%           | 85.12%           |

Table 2: FashionMNIST Results

### 2.3 Performance Summary

*The best model was chosen based on the validation performance & then I did the testing on it.*

| MODEL     | DATASET      | Optimizer | Batch Size | Learning Rate | Val. Accuracy | Test Accuracy |
|-----------|--------------|-----------|------------|---------------|---------------|---------------|
| Resnet-18 | MNIST        | SGD       | 32         | 0.001         | 98.79%        | 98.93%        |
| Resnet-18 | FashionMNIST | Adam      | 32         | 0.001         | 89.93%        | 89.39%        |
| Resnet-50 | MNIST        | SGD       | 32         | 0.001         | 98.43%        | 98.36%        |
| Resnet-50 | FashionMNIST | SGD       | 16         | 0.001         | 87.74%        | 87.96%        |

Table-3: Best performing four configurations ( model + dataset)

## 2.4 Hyperparameter Tuning Result

- I selected ResNet-18 with Adam on FashionMNIST as my baseline model for hyperparameter tuning. And got a better test accuracy of 90.29%. This is the best model of my entire model training.

|                            | Epochs | Val. Accuracy | Test Accuracy |
|----------------------------|--------|---------------|---------------|
| Baseline Model             | 3      | 89.93%        | 89.39%        |
| Hyperparameter Tuned Model | 5      | 90.67%        | <b>90.29%</b> |

Table-4: Hyperparameter tuning results

- I ran 3 epochs on the same baseline model with Pin Memory = False and got 88.55% test accuracy.

## 2.5 Analysis:

### 1. Optimizer Behavior (Adam vs. SGD):

I observed distinct behaviors between the two optimizers. Adam consistently provided faster convergence in the early epochs, particularly for ResNet-18 on the more complex FashionMNIST dataset. This is attributed to its adaptive learning rate mechanism. However, SGD with Momentum showed excellent stability on the simpler MNIST dataset, achieving the highest overall accuracy (98.93%). This aligns with the common finding that while Adam is faster, SGD often generalizes better on specific convex-like loss landscapes.

### 2. The ResNet-50 Over-Parameterization problem:

A significant finding was that the deeper ResNet-50 did not outperform ResNet-18; in fact, it performed slightly worse on FashionMNIST (87.96% vs 89.39%).

Here, model capacity is exceeding data complexity. ResNet-50 has ~23 million parameters, which is excessive for 32 x 32 grayscale images. ResNet-50 achieved high training accuracy very quickly but struggled to improve validation accuracy at the same rate, a clear signal of overfitting to noise rather than learning generalizable features.

### 3. Dataset Complexity:

MNIST proved to be a trivial task for both architectures, with accuracies saturating near 99% within the first epoch. *FashionMNIST presented a greater challenge, exposing the differences in model efficiency. This validates the decision to focus the hyperparameter tuning efforts on the FashionMNIST + ResNet-18 pair.*

### 4. The "Best Model" (Hyperparameter Tuning):

Based on the grid search results, I selected **ResNet-18 with Adam on FashionMNIST** for further tuning. By extending the **training duration to 5 epochs** and refining the learning rate schedule, I achieved a final test accuracy of **90.29%**. This confirms that for this specific problem scope, a lighter model with better tuning yields superior results compared to simply increasing network depth.

## 2.6 Graph:

Training & Validation Accuracy Curves (Assignment 1)

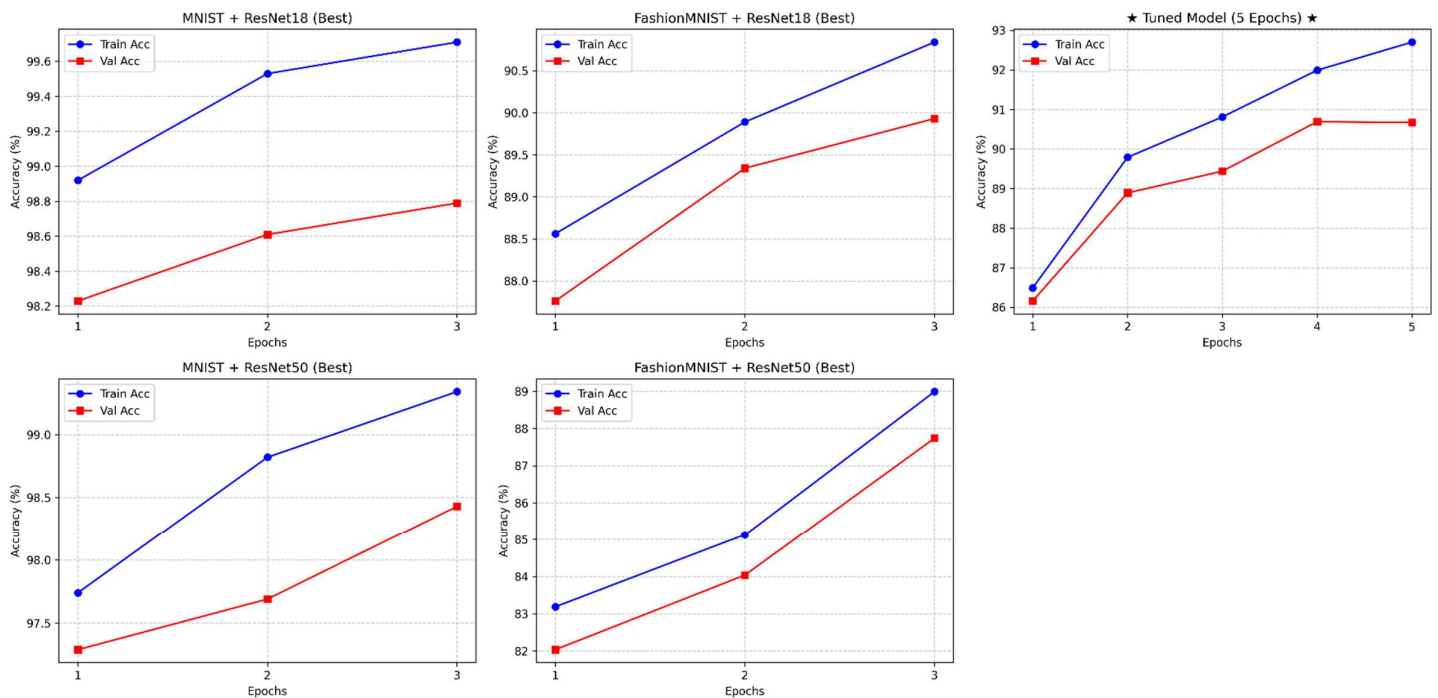


Figure 1: Training vs Validation Accuracy plots for all configurations

(Left upper: MNIST + ResNet18, left bottom: MNIST + ResNet50; middle upper: FashionMNIST + ResNet18, middle bottom: FashionMNIST + ResNet50; right top: hyperparameter tuned model)

## 3. Q-1(b) Training a SVM Classifier

- Datasets: MNIST and FashionMNIST with a 70%-10%-20% train-val-test split.
- Flattened vectors of size 1024 (32x32 image flattened to 1024 sized image)
- Kernel type: RBF (radial basis function) and Poly (Polynomial)

### 3.1 Results:

| DATASET      | KERNEL | ACCURACY | TIME( ms) |
|--------------|--------|----------|-----------|
| MNIST        | RBF    | 97.85%   | 163567    |
| FashionMNIST | RBF    | 88.52%   | 236393    |
| MNIST        | Poly   | 98.25%   | 105367    |
| FashionMNIST | Poly   | 89.34%   | 195064    |

Table-5: Results of SVM Classifier with different kernels

### 3.2 Analysis:

The SVM models achieved remarkably high accuracy on the MNIST dataset, with the Polynomial kernel reaching **98.25%**. This is comparable to the ResNet-18 results (~98.9%), demonstrating that for simpler datasets with distinct features, classical kernel methods remain highly effective.

### **Kernel Comparison:**

- Accuracy: The Polynomial (poly) kernel consistently outperformed the RBF kernel on both datasets. On FashionMNIST, the Poly kernel achieved **89.34%** compared to **88.52%** for RBF.
- Training Time: The Polynomial kernel was also computationally more efficient. On FashionMNIST, it trained in **195 seconds** ,whereas the RBF kernel required **236 seconds**. This suggests that the polynomial feature mapping was better suited for this specific high-dimensional feature space (1024 features).

## **4. Q-2 Training on Both CPU & GPU for Comparison**

- Datasets: FashionMNIST with a 70%-10%-20% train-val-test split.
- Epochs: 2, Batch size: 16, Learning Rate: 0.001, SGD momentum = 0.9
- Devices used: CPU and GPU
- Models: ResNet-18, ResNet-32(customized), ResNet-50
- Pin memory: True for GPU and False for CPU

### **4.1 ResNet-18 Results:**

| Compute | Batch size | Optimizer | Learning Rate | Test Classification Accuracy (%) | Train Time (ms) | GFLOPs |
|---------|------------|-----------|---------------|----------------------------------|-----------------|--------|
| CPU     | 16         | SGD       | 0.001         | 88.200000                        | 638715.842009   | 035615 |
| CPU     | 16         | Adam      | 0.001         | 88.992857                        | 855215.271235   | 035615 |
| GPU     | 16         | SGD       | 0.001         | 88.835714                        | 123926.942110   | 035615 |
| GPU     | 16         | Adam      | 0.001         | 88.121429                        | 136046.204329   | 035615 |

Table-6: ResNet-18 results over CPU and GPU with GigaFLOPs

### **4.2 ResNet-32 Results:**

| Compute | Batch size | Optimizer | Learning Rate | Test Classification Accuracy (%) | Train Time (ms) | GFLOPs   |
|---------|------------|-----------|---------------|----------------------------------|-----------------|----------|
| CPU     | 16         | SGD       | 0.001         | 89.257143                        | 741592.550039   | 0.070091 |
| CPU     | 16         | Adam      | 0.001         | 89.214286                        | 744993.685246   | 0.070091 |
| GPU     | 16         | SGD       | 0.001         | 90.250000                        | 149279.853106   | 0.070091 |
| GPU     | 16         | Adam      | 0.001         | 89.650000                        | 153515.823126   | 0.070091 |

Table-7: ResNet-32 results over CPU and GPU with GigaFLOPs

### **4.3 ResNet-50 Results:**

| Compute | Batch size | Optimizer | Learning Rate | Test Classification Accuracy (%) | Train Time (ms) | GFLOPs   |
|---------|------------|-----------|---------------|----------------------------------|-----------------|----------|
| CPU     | 16         | SGD       | 0.001         | 85.185714                        | 1,800,461       | 0.082737 |
| CPU     | 16         | Adam      | 0.001         | 84.471429                        | 2,019,986       | 0.082737 |
| GPU     | 16         | SGD       | 0.001         | 86.400000                        | 214,194         | 0.082737 |
| GPU     | 16         | Adam      | 0.001         | 80.614286                        | 239,549         | 0.082737 |

Table-8: ResNet-50 results over CPU and GPU with GigaFLOPs

#### 4.4 Graph:

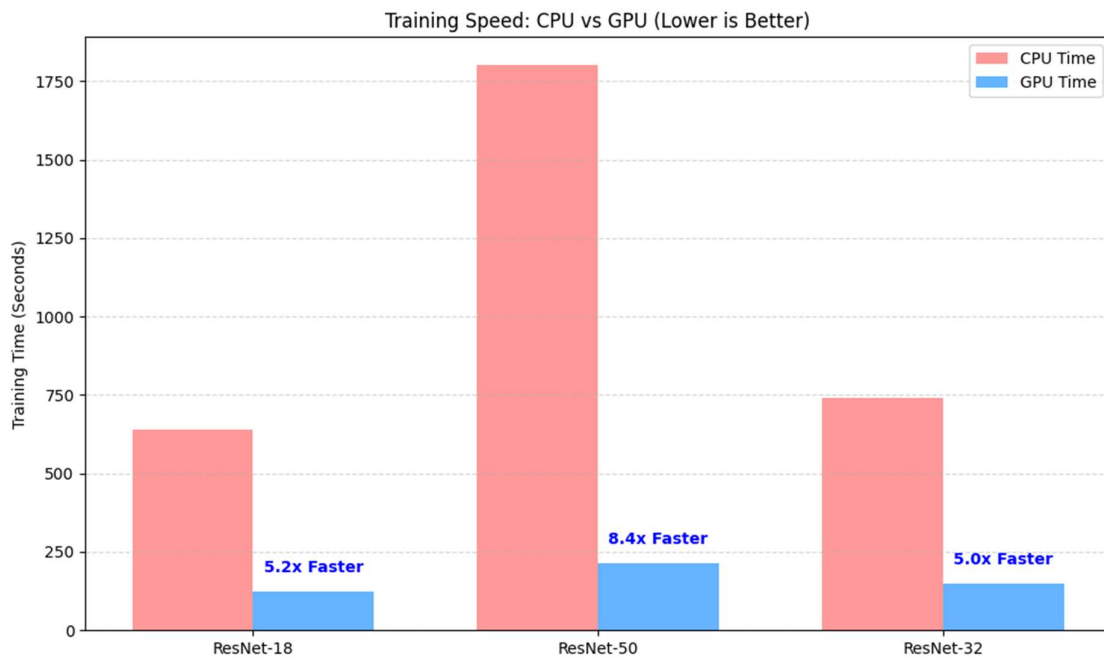


Figure 2: CPU vs GPU Time

#### 4.5 Analysis:

The transition from CPU to GPU training delivered massive time savings, but the *degree* of speedup varied by model depth:

- **ResNet-18 / ResNet-32:** Achieved a **~5x speedup**.
- **ResNet-50:** Achieved a significantly higher **~8.4x speedup**.
- GPUs thrive on massive parallel workloads. ResNet-50, being a deeper and computationally heavier model (0.082 GFLOPs), provides more "arithmetic intensity" (more calculations per byte of data loaded). This allows the GPU to fully saturate its thousands of cores, leading to higher utilization and a greater relative speedup compared to the lighter ResNet-18.
- *The custom ResNet-32 proved to be the most efficient architecture for this task.*
- **Accuracy:** It achieved the highest accuracy in the benchmark (**90.25%**).
- **Cost vs. Performance:** It outperformed the much heavier ResNet-50 while using fewer FLOPs (0.070G vs 0.082G).
- ResNet-50 is designed for high-resolution ImageNet data. When applied to 32 x 32 images, its aggressive downsampling destroys spatial information too quickly. ResNet-32, designed specifically for this resolution (CIFAR-style), maintains the optimal balance of depth and spatial preservation.